

1) Merge Sort (with Output)

```
#include <stdio.h>

void merge(int a[], int l, int m, int r) {
    int i=l, j=m+1, k=0, temp[1000];
    while(i<=m && j<=r)
        temp[k++]=(a[i]<a[j])?a[i++]:a[j++];
    while(i<=m) temp[k++]=a[i++];
    while(j<=r) temp[k++]=a[j++];
    for(i=l, k=0; i<=r; i++, k++)
        a[i]=temp[k];
}

void mergesort(int a[], int l, int r) {
    if(l<r) {
        int m=(l+r)/2;
        mergesort(a, l, m);
        mergesort(a, m+1, r);
        merge(a, l, m, r);
    }
}

int main() {
    int n, a[1000], i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);

    mergesort(a, 0, n-1);

    printf("Sorted array:\n");
    for(i=0; i<n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Sample Output

```
Enter number of elements: 6
Enter elements:
8 4 2 9 1 5
Sorted array:
1 2 4 5 8 9
```

2) N-Queen (with Chess Board Output)

```
#include <stdio.h>
#include <stdlib.h>

int x[20], n;

int place(int k, int i) {
    for(int j=1; j<k; j++)
        if(x[j]==i || abs(x[j]-i)==abs(j-k))
            return 0;
    return 1;
}

void queen(int k) {
    int i, j;

    for(i=1; i<=n; i++) {
        if(place(k, i)) {
            x[k]=i;

            if(k==n) {
                printf("\nSolution:\n");
                for(i=1; i<=n; i++) {
                    for(j=1; j<=n; j++) {
                        if(x[i]==j)
                            printf("Q ");
                        else
                            printf(". ");
                    }
                    printf("\n");
                }
            }
            else
                queen(k+1);
        }
    }
}

int main() {
    printf("Enter value of N: ");
    scanf("%d", &n);

    queen(1);

    return 0;
}
```

Sample Output (N = 4)

Enter value of N: 4

Solution:

```
. Q . .
. . . Q
Q . . .
. . Q .
```